Data Types & Coercion

12 min read · Updated 17 Jul 2026 · Topic 5 of 23 · Bookmark · Completed

Data types and coercion are the single most common source of JavaScript trick questions, because the language will silently convert almost anything into almost anything else. Interviewers use it to test whether you actually understand what happens under the hood or just memorised outputs. This lesson covers every type, the `typeof` quirks, value-vs-reference semantics, the full `==` coercion algorithm, and the famous "what does this print" traps — every output here was run in Node before being written down.

The type system: 7 primitives + object

JavaScript has exactly **7 primitive types** and one non-primitive type, `object`. Primitives are immutable and compared by value; objects are mutable and compared by reference.

- `string`, `number`, `boolean`, `undefined`, `null`, `symbol` (ES6), `bigint` (ES2020) — the 7 primitives.
- `object` — everything else: plain objects, arrays, functions, dates, regexes, `Map`, `Set`. Functions are a *callable* object subtype.

`typeof` and its two famous quirks

`typeof` returns a lowercase string naming the type. It is the standard runtime type check, but it has two answers everyone must know cold.

JS Copy

```
typeof 42            // "number"
typeof "hi"          // "string"
```

```
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof 10n           // "bigint"
typeof Symbol("s")   // "symbol"
typeof {}            // "object"
typeof []            // "object"   (arrays are objects — use Array.isArray
typeof null          // "object"   ⟵ the famous bug
typeof function(){}  // "function" ⟵ functions get their own answer
```

`typeof null === "object"` is a **bug that has existed since 1995** and can never be fixed without breaking the web: in the original implementation values were tagged by a type bit-pattern, and `null` was the null pointer `0x00`, which shared the object tag. To actually test for `null`, compare directly: `value === null`. And `typeof function(){}` returns `"function"` even though functions *are* objects — this is a deliberate convenience so you can feature-test callables.

Because `typeof []` and `typeof null` are both `"object"`, a robust "is it a real object" check is: `typeof x === "object" && x !== null && !Array.isArray(x)`.

Value types vs reference types

Primitives are **copied by value** — assigning one variable to another duplicates the data. Objects are **copied by reference** — the variable holds a pointer, so two variables can point at the *same* object.

JS Copy

```
let a = 10;
let b = a;       // b gets a COPY
b = 20;
console.log(a); // 10  (unaffected)

let obj = { x: 1 };
let ref = obj;  // ref points at the SAME object
ref.x = 2;
console.log(obj.x); // 2  (both see the mutation)
```

This is why equality of objects checks *identity*, not contents. Two objects with identical shape are never `===`, but two references to the same object are.

JS Copy

```
console.log({} === {});                  // false  (two different objects)
console.log([1, 2] === [1, 2]);          // false  (different arrays)
const p = { x: 1 };
console.log(p === p);                    // true   (same reference)
console.log([1, 2].join() === [1, 2].join()); // true (comparing STRINGS n
```

The same reference semantics apply to function arguments: passing an object into a function lets that function mutate the caller's object, but reassigning the parameter does not affect the caller.

`==` vs `===`

`===` (strict equality) compares type **and** value with no conversion: if the types differ it is immediately `false`. `==` (loose equality) applies a coercion algorithm first, which is where the surprises live.

Here is the loose-equality algorithm at a useful depth (the abstract equality comparison, simplified but accurate):

1. If both operands have the **same type**, compare with `===`.
2. `null == undefined` is `true`; neither loosely equals anything else.
3. **number vs string** → convert the string to a number, then compare.
4. **boolean vs anything** → convert the boolean to a number (`true` → `1`, `false` → `0`), then re-compare.
5. **object vs number/string/bigint/symbol** → convert the object to a primitive (via `valueOf` / `toString`), then re-compare.
6. bigint vs number/string compares mathematical values.
7. Anything else is `false`.

The critical takeaways: `false` becomes `0`, `""` becomes `0`, `[]` becomes `""` (then `0`), and `null` / `undefined` only equal each other.

```
console.log(0 == "");         // true   ('' → 0)
console.log(0 == "0");        // true   ('0' → 0)
console.log("" == "0");       // false  (same type, plain string compare)
console.log(false == "");     // true   (false → 0, '' → 0)
console.log(false == "0");    // true   (both → 0)
console.log(null == 0);       // false  (null only == undefined)
console.log([] == "");        // true   ([] → '')
console.log([] == 0);         // true   ([] → '' → 0)
console.log([0] == false);    // true   (false→0, [0]→'0'→0)
```

When `==` is acceptable: the one idiomatic, defensible use is `x == null`, which is `true` for *both* `null` and `undefined` and `false` for everything else (including `0`, `""`, `false`). It is a clean nullish check. Everywhere else, prefer `===` — `==` is banned by most linters for good reason.

JS Copy

```
console.log(null == undefined);  // true
console.log(null === undefined); // false
function greet(name) {
  if (name == null) name = "friend"; // catches null AND undefined, not ""
  return "hi " + name;
}
```

Coercion rules: to-number, to-string, to-boolean

Three abstract operations drive nearly all coercion. Know their edge cases.

ToNumber — strings are trimmed then parsed (empty/whitespace → `0`); `null` → `0` but `undefined` → `NaN`; `true` → `1`, `false` → `0`; arrays convert via their string form (so `[]` → `0`, `[5]` → `5`, but `[1,2]` → `NaN`); plain objects → `NaN`.

JS Copy

```
console.log(Number(""));        // 0
console.log(Number("  "));      // 0    (whitespace only)
```

```
console.log(Number(undefined)); // NaN
console.log(Number([]));        // 0
console.log(Number([5]));       // 5
console.log(Number([1, 2]));    // NaN
console.log(Number({}));        // NaN
```

ToString — `null` → `"null"`, `undefined` → `"undefined"`, arrays join with commas, plain objects become `"[object Object]"`.

JS Copy

```
console.log(String(null));    // "null"
console.log(String([1,2,3])); // "1,2,3"
console.log(String({}));      // "[object Object]"
```

ToBoolean — everything is truthy **except** the 8 falsy values. Memorise the falsy list: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. That's it — everything else (including `"0"`, `"false"`, `[]`, `{}`, and every function) is truthy.

JS Copy

```
console.log(Boolean(""));    // false
console.log(Boolean("0"));   // true   (non-empty string!)
console.log(Boolean([]));    // true   (empty array is truthy!)
console.log(Boolean({}));    // true
console.log(Boolean(NaN));   // false
```

The `+` operator is special

Every arithmetic operator (`-`, `*`, `/`, `%`) coerces both sides to **numbers**. But `+` is overloaded: if **either operand is a string** (after object-to-primitive conversion), it does **string concatenation**; otherwise it adds. This single rule explains most trick questions.

JS Copy

```
console.log("5" - 2);   // 3     (minus forces numbers)
console.log("5" * "2"); // 10    (both → numbers)
console.log(1 + 2 + "3"); // "33" (left-to-right: 3, then "33")
console.log("1" + 2 + 3); // "123"
```

Famous trick outputs (all node-verified)

These are the classic whiteboard questions. Each output below was executed in Node.

JS Copy

```
console.log([] + []);   // ""              (both arrays → "" , concat)
console.log([] + {});   // "[object Object]"  ("" + "[object Object]")
console.log(1 + "1");   // "11"
console.log("5" - 2);   // 3
console.log(null == undefined); // true
console.log(NaN === NaN);       // false   (NaN is never equal to anything
console.log(0.1 + 0.2);         // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

`{} + []` is the trickiest because it is **parse-dependent**:

JS Copy

```
// Expression position (assigned, passed, or logged): {} is an OBJECT
console.log({} + []);        // "[object Object]"
const r = {} + [];
console.log(r);              // "[object Object]"

// Statement position (start of a line in a browser/REPL): {} is an empty
// so it becomes:  {}          ← empty block, does nothing
//                 +[]         ← unary plus on []  → 0
console.log(+[]);            // 0   (this is what the "{} + []" REPL trick
```

a leading `{}` is parsed as an empty code block and only `+[]` remains. Node's REPL auto-wraps the line, so it also prints `"[object Object]"`.

`0.1 + 0.2 !== 0.3` is not a JS bug — it is **IEEE-754 double-precision floating point**, the same in Python, Java, and C. `0.1` and `0.2` have no exact binary representation, so their sum drifts by ~5.5e-17. Compare floats with a tolerance (epsilon): `Math.abs(a - b) < Number.EPSILON`.

`null` vs `undefined`

Both are falsy and both mean "no value", but they signal different intents. `undefined` is the language's default "absence"; `null` is a programmer's explicit "intentionally empty".

Aspect	`undefined`	`null`
Meaning	value was never assigned	intentional "no value"
`typeof`	`"undefined"`	`"object"` (the bug)
Set by	the engine (missing var, param, return, property)	the developer, explicitly
`Number(x)`	`NaN`	`0`
JSON	key is **omitted** from `JSON.stringify`	serialised as `null`
Default params	triggers the default	does **not** trigger the defaul
`x == null`	`true`	`true`
`x === null`	`false`	`true`

`NaN`, `isNaN` vs `Number.isNaN`, and `Object.is`

- `isNaN(x)` (global) coerces its argument to a number first, so `isNaN("foo")` is `true` — misleading, because `"foo"` was never `NaN`, it just became `NaN` on conversion.
- `Number.isNaN(x)` (ES6) does **no coercion** — it returns `true` only for the actual `NaN` value. Prefer it.

JS Copy

```
console.log(NaN === NaN);        // false
console.log(NaN !== NaN);        // true  (the manual NaN test)
console.log(isNaN("foo"));       // true  (coerces "foo" → NaN)
console.log(Number.isNaN("foo"));// false (no coercion, "foo" isn't NaN)
console.log(isNaN("12"));        // false
console.log(Number.isNaN(NaN));  // true
```

`Object.is(a, b)` is "SameValue" equality — like `===`, but with two deliberate differences: `Object.is(NaN, NaN)` is `true`, and `Object.is(-0, 0)` is `false` (whereas `-0 === 0` is `true`). Use it when `-0` vs `+0` or `NaN` identity actually matters.

JS Copy

```
console.log(Object.is(NaN, NaN)); // true   (=== gives false)
console.log(Object.is(-0, 0));    // false  (=== gives true)
console.log(-0 === 0);            // true
```

Common interview questions

Q: How many primitive types does JavaScript have, and what are they? Seven: `string`, `number`, `boolean`, `undefined`, `null`, `symbol`, `bigint`. Everything else is an `object`, and functions are a callable object subtype.

Q: Why does `typeof null` return `"object"`? A legacy bug from the first JS engine, where values were tagged by a type field and `null` (the null pointer, all-zero bits)

Q: What is the difference between `==` and `===`? `===` compares type and value with no conversion. `==` runs the abstract-equality algorithm: `null == undefined`, numbers vs strings convert the string to a number, booleans convert to numbers, and objects convert to primitives before comparing. Use `===` everywhere except the deliberate `x == null` nullish check.

Q: Explain `[] == ![]`. It is `true`. `![]` evaluates first — `[]` is truthy so `![]` is `false`. Now `[] == false`: the boolean becomes `0`, the array becomes `""` then `0`, so `0 == 0` is `true`. It's a favourite because it looks like a thing compared to its own negation.

Q: Why is `0.1 + 0.2 !== 0.3`? Doubles are binary IEEE-754; `0.1` and `0.2` aren't exactly representable, so the sum is `0.30000000000000004`. Compare with a tolerance: `Math.abs(a - b) < Number.EPSILON`.

Q: `isNaN` vs `Number.isNaN`? `isNaN` coerces its argument first (`isNaN("foo") === true`), so it answers "is this NaN *after* conversion". `Number.isNaN` does no coercion and returns `true` only for the genuine `NaN` value. Prefer `Number.isNaN`.

Q: How do you copy an object vs a primitive? Primitives copy by value automatically (`let b = a`). Objects copy by reference, so `let b = a` shares the same object; for a real copy use a spread `{ ...a }` / `[ ...a]` (shallow) or `structuredClone(a)` (deep).

Gotchas / traps

`typeof NaN` is `"number"`. `NaN` is a numeric value, so this catches people who expect `"NaN"`.

Empty array and empty object are truthy, but `[] == false` is true. `Boolean([])` is `true`, yet `[] == false` is also `true` — because `==` converts `[]` to `""` to `0` and `false` to `0`. Truthiness (used by `if`) and loose equality use *different* conversions. Never rely on `== true` / `== false`.

`Number([1,2])` is `NaN` but `Number([5])` is `5`. A one-element array stringifies to `"5"` (numeric), a multi-element array to `"1,2"` (not numeric). This surprises people who expect arrays to always coerce to `NaN` or to their length.

Comparison operators are non-associative traps.

JS Copy

```
console.log(1 < 2 < 3); // true   (1<2 → true → 1, then 1<3)
console.log(3 > 2 > 1); // false  (3>2 → true → 1, then 1>1 is false)
```

`"b" + "a" + +"a" + "a"` **is** `"baNaNa"`. The `+ +"a"` is a unary plus on `"a"`, which is `NaN`, and concatenation stringifies it to `"NaN"`.

JS Copy

```
console.log("b" + "a" + + "a" + "a"); // "baNaNa"
```

Large integers lose precision — use `BigInt`. Beyond `Number.MAX_SAFE_INTEGER` (2^53 - 1) integers can't be represented exactly:

JS Copy

```
console.log(9007199254740992 === 9007199254740993); // true  (both round t
console.log(9007199254740993n === 9007199254740993n); // true (BigInt is e
```

`typeof typeof anything` **is always** `"string"`. The inner `typeof` returns a string, and `typeof "..."` is `"string"` — a cute misdirection: `typeof typeof 1 === "string"`.

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```
console.log("5" + 2 - 1);
```

- ◯ 6

- "521"
- NaN

Check answer

2. What does this log?

JAVASCRIPT Copy

```
console.log(1 < 2 < 3);
console.log(3 > 2 > 1);
```

- true
  true
- false
  false
- true
  false
- false
  true

Check answer

3. What is the result of `[] == ![]`, and why?

- true — `![]` is false, then `[]` coerces to "" then 0 and false to 0, so 0 == 0
- false — an array is never loosely equal to a boolean
- true — `[]` is truthy so `![]` is true, and truthy == truthy

Check answer

Previous
Javascript Interview Preparation questions

Next
Scope, Hoisting & the TDZ

LEARN MORE

ARTICLES & DOCS

- JavaScript data types (MDN)
  MDN
- Equality comparisons & sameness
  MDN

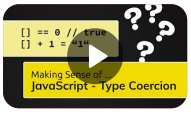
VIDEOS



- Type conversion & coercion
  YouTube



- Coercion & type conversion, simplified
  YouTube



- How type coercion works
  YouTube

Hand-picked references for this lesson.